Informe post-incidente — TP Unidad 6

Food Store · Mantenimiento, respaldo y recuperación

Estado: versión final del 2026-10-28. Integra las evidencias de las cuatro partes (`docs/i1_*` a `docs/i4_*`). El PDF se genera con `python scripts/generar_informe_pdf.py`.

Entorno. PostgreSQL 17.11. El puerto por defecto del equipo es 5433 (máquina de Matías); en la máquina donde se ejecutaron el respaldo y el simulacro (FACU) el servidor escucha en 5432 y los scripts reciben `-Port 5432` explícito. Base `foodstore` de 102 MB (11 categorías, 20.000 usuarios, 50.000 productos, 200.000 pedidos y 400.000 detalles). Base de ensayo `foodstore_restore_test`, descartable. Autenticación por `%APPDATA%\postgresql\pgpass.conf`; ninguna contraseña en scripts ni en el repositorio. Evidencia de la reconstrucción: `docs/i1_reconstruccion_20261028_1630.txt`.

Respaldo utilizado. `foodstore_20261028_2008.dump`, generado por `.\scripts\backup_foodstore.ps1 -Port 5432` el 2026-10-28 entre las 20:08:27 y las 20:08:38. Tamaño: 9.378.143 bytes (8,94 MB). SHA-256: `DA38B10B7C0F87D3EF3D018809543E80406B08A20EA702E3933B96C110BE096D`, recalculado de forma independiente con `Get-FileHash` y coincidente con el del log. `pg_restore -l` lo lista sin error con **33 objetos**: 5 tablas, 5 secuencias, 5 bloques `TABLE DATA`, 5 `SEQUENCE SET`, 6 PK/UNIQUE, 3 índices y 4 claves foráneas. Evidencia: `docs/i2_backup_20261028_2008.txt`.

Simulacro. Comando `.\scripts\restore_test.ps1 -Port 5432`, que restaura en `foodstore_restore_test` (la base `foodstore` no se toca en ningún paso) con `pg_restore --no-owner --no-privileges --exit-on-error`. Inicio 20:38:30; T0 = 20:38:31,651, inmediatamente antes del `createdb`; T1 = 20:38:48,395, al terminar la validación. Resultado `PASS` y `EXITCODE=0`. Después del ensayo se verificó aparte que `foodstore` seguía con los mismos conteos y el mismo `PASS`. Evidencia: `docs/i4_simulacro_20261028_2038.txt`.

RTO y RPO.

- **RTO previsto:** 300 s como máximo (5 minutos). **Observado: 16,7 s**, desde el `createdb` hasta el fin de las validaciones.
- **RPO previsto:** 24 h como máximo, porque el plan es un respaldo diario a las 03:00. **Observado: 29 min 54 s** (1.794,5 s), medido como la antigüedad del `.dump` al iniciar el cronómetro.
- El RPO observado depende del momento del simulacro: se corrió 29 minutos después del respaldo, así que **no representa el peor caso del plan**. El peor caso del esquema actual es de hasta 24 h (todo lo cargado entre las 03:00 y el incidente); bajarlo exige archivado WAL + PITR.

Validaciones. Sobre `foodstore_restore_test`, contra los conteos registrados antes del respaldo. El chequeo consolidado (`sql/validaciones_restore_check.sql`) devuelve una sola línea: `PASS`.

- **Conteos:** 11 categorías, 20.000 usuarios, 50.000 productos, 200.000 pedidos y 400.000 detalles, idénticos al pre-respaldo (5 de 5 en `OK`).
- **Productos inválidos** (`precio_lista <= 0` o `stock < 0`): 0.
- **Detalles inválidos** (`cantidad <= 0` o `subtotal < 0`): 0.
- **Claves foráneas huérfanas** (pedido a usuario, producto a categoría, detalle a pedido y detalle a producto): 0.
- **Consistencia de totales** (`pedido.total = SUM(detalle_pedido.subtotal)`): 0 pedidos descuadrados sobre 200.000.
- **Secuencias restauradas** (`last_value >= max(id)`): las 5 al día (11, 20.000, 50.000, 200.000 y 400.000).
- **Prueba negativa:** al romper el `CHECK` de `producto.precio_lista` y dejar una fila en `-1`, el chequeo devuelve `FALLA:productos_invalidos=1`; al reconstruir la base vuelve a `PASS`.

Problema real encontrado. Cada parte dejó al menos un error concreto, detectado al ejecutar y no al leer:

- **I2 (respaldo):** la `foodstore` de la máquina donde se regeneró la evidencia estaba vacía y sin la tabla `usuario`, así que el respaldo habría sido inútil. Se detectó con los conteos previos, se reconstruyó la base y recién entonces se respaldó. Además, la primera evidencia (`i2_backup_20261028_1656.txt`) mostraba `Dump generado` antes de la verificación con `pg_restore -l`; se corrigió el orden del script y se regeneró.
- **I3 (mantenimiento):** la reconstrucción de la base hacía `DROP TABLE ... CASCADE`, que habría arrastrado objetos de TP5/TP7 fuera del alcance; se reemplazó por una guardia que aborta sin borrar nada. En el health check, una columna de tamaño de índices mal calculada (`pg_indexes_size - pg_relation_size`) daba valores negativos y se eliminó.
- **I4 (restauración):** la prueba negativa se planteó con un `INSERT` de un producto con `precio_lista = -1`; eso cambiaba los conteos y el chequeo devolvía `FALLA:conteos(producto=50001/50000)` en lugar del control que se quería demostrar. Se cambió a un `UPDATE` sobre una fila existente y el resultado pasó a ser `FALLA:productos_invalidos=1`. En los escenarios de falla se comprobó además que un dump truncado **no** se detecta con `pg_restore -l`, porque el TOC del formato custom está al inicio del archivo: el restore falla recién al leer los datos y deja una base parcial (5 tablas, sólo `categoria` con 11 filas, 14 MB) que el chequeo de conteos rechaza con `FALLA:conteos`.

Mejora concreta. El respaldo quedó programado a las 03:00 con el Programador de tareas de Windows, con retención de 7 días (`-Retention 7`) y verificación obligatoria del dump con `pg_restore -l` antes del rename. Sobre esa base se propone:

- **Ensayo de restauración semanal automatizado** con `restore_test.ps1` en una tarea programada, que deje `EXITCODE=1` y alerta si la validación no devuelve `PASS`.
- **Health check diario de I3 como control previo al respaldo**, para no respaldar una base degradada.
- **Archivado WAL + PITR** para llevar el RPO de 24 h a minutos: es la limitación real del plan actual. Un RPO de 24 h sólo es aceptable si el negocio acepta perder un día completo de pedidos, algo que para un food store con ventas online no parece razonable.

Intervención de OpenCode.

- **Matías (I1):** generó la especificación, las reglas de `AGENTS.md` y el script de reconstrucción reproducible con `setseed`. Se revisó contra la consigna y se corrigieron los nombres de columnas al esquema real (`usuario`, `precio_lista`, `subtotal`).
- **Santi (I2):** generó `backup_foodstore.ps1`. Se auditó línea por línea (sin `VACUUM`, sin `DROP`, sin contraseñas, `.parcial` + rename) y se probaron el camino de error y la retención antes de aceptarlo.
- **Facu Deseff (I3):** generó `health_check.sql` y `maintenance.sql`. Se eliminó una métrica inventada y se agregó un umbral absoluto de tuplas muertas después de ver la primera salida.
- **Facu Ramirez (I4):** generó `restore_test.ps1` y las dos validaciones SQL. Se corrigió la prueba negativa (`UPDATE` en vez de `INSERT`) y se agregaron tres escenarios de falla. El detalle completo de prompts y verificaciones está en `BITACORA.md`.

Evidencias.

- **I1:** `docs/i1_reconstruccion_20261028_1630.txt`
- **I2:** `docs/i2_backup_20261028_2008.txt` (vigente) y `docs/i2_backup_20261028_1656.txt` (obsoleta, se conserva sin editar)
- **I3:** `docs/i3_health_antes_20261028_1816.txt`, `docs/i3_churn_20261028_1816.txt`, `docs/i3_health_con_muertas_20261028_1816.txt`, `docs/i3_maintenance_20261028_1816.txt` y `docs/i3_health_despues_20261028_1816.txt`
- **I4:** `docs/i4_simulacro_20261028_2038.txt` y `docs/i4_prueba_negativa_20261028_2038.txt`